

Amortized eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Topologies

First Author
firstname@university.edu
University Name
City, State, Country

Second Author
secondname@university.edu
University Name
City, State, Country

Third Author
thirdname@university.edu
University Name
City, State, Country

Abstract

Field-Programmable Gate Arrays (FPGAs) offer essential hardware flexibility through programmable logic and interconnects. Embedded FPGAs (eFPGAs) extend this capability by integrating reconfigurable fabrics directly into fixed-silicon Systems-on-Chip (SoCs) or ASICs, enabling post-fabrication functional updates and robust IP security. Hence, discovering optimized layouts for these reconfigurable fabrics is essential to fully realize their architectural potential. We train a single graph-and-grid-conditioned Reinforcement Learning (RL) policy across various benchmark circuits to place heterogeneous blocks (H-blocks, such as DSPs and BRAMs) and select the optimal fabric aspect ratio for the eFPGA architectures. We thereby generate layouts jointly optimized for Power, Performance, and Area (PPA). To demonstrate generalizability, we evaluate the trained RL policy on benchmark circuits completely unseen during training, producing highly efficient zero-shot layouts. Across three random seeds, the policy achieves up to a 58.73% zero-shot ADP reduction on unseen benchmark circuits relative to traditional island-style architectures.

CCS Concepts

• **Hardware** → **Physical design (EDA)**; *Reconfigurable logic and FPGAs*; • **Computing methodologies** → *Reinforcement learning*.

Keywords

FPGA, Reinforcement Learning (RL), Graph Convolutional Networks (GCN), Convolutional Neural Networks (CNN), Macro Placement, Reconfigurable Computing

ACM Reference Format:

First Author, Second Author, and Third Author. 2027. Amortized eFPGA Layout Search via Reinforcement Learning: Zero-Shot Generalization Across Circuit Topologies. In *Proceedings of Asia and South Pacific Design Automation Conference (ASP-DAC '27)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

A generic FPGA inherently over-provisions hardware resources for any single application, carrying extra routing and heterogeneous blocks (*H-blocks*: DSP slices, BRAMs) that the deployed netlist never fully utilizes. Sizing an eFPGA fabric for one specific netlist recovers

much of this overhead. The product of routing area, critical-path delay, and total power (**ADP** throughout) directly quantifies this overhead; minimizing this metric benefits every eFPGA application [4, 20].¹

Prior efforts optimize fabric layouts on a per-design basis using computationally expensive search heuristics like Genetic Algorithm (GA) [20] or NSGA-II [4]. These approaches restart the search from scratch for every new circuit, transferring no knowledge across runs and incurring the full computational search cost each time.

We train a single graph-and-grid-conditioned RL policy to determine H-block placement and fabric aspect ratio, addressing a domain where computationally expensive per-instance GA search previously served as the only option. Unlike RL methods [5, 11] that assign blocks to a pre-fabricated grid, our policy directly constructs the fabric layout itself. After jointly training on 11 circuits evaluated via the standard VTR flow, we apply a single model checkpoint zero-shot to unseen circuits, including fabrics up to $3.4\times$ the maximum training pool area. We train only once, after which evaluating a new circuit requires just a single policy rollout. We compare against a from-scratch GA under an identical oracle, metric, and baseline: the GA retains a higher absolute ADP reduction on some circuits, but each of its generations requires its own VTR call, a cost that grows with circuit size. The zero-shot policy requires a single VTR call per circuit, so its relative advantage grows on the larger circuits where the GA's search cost is highest (Section 5.3).

We contribute:

- A graph-and-grid-conditioned RL policy that *constructs* the eFPGA fabric itself, placing H-blocks and selecting fabric aspect ratio, rather than assigning blocks to an already-fabricated grid as prior RL placement methods do.
- An amortized alternative to per-instance GA/NSGA-II search: training cost is incurred once and reused through a single rollout per new circuit, whereas evolutionary search restarts from scratch for every design.
- A multi-modal feature extractor combining a GNN encoding of the netlist graph with a CNN encoding of the occupancy grid, giving the policy a fixed action space and the inductive bias for cross-topology transfer across varying netlist sizes and H-block counts.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ASP-DAC '27,

© 2027 Copyright held by the owner/author(s). Publication rights licensed to ACM.

¹GOLDS [20] uses the same abbreviation, ADP, for a different, two-term (logical_area $\times$ delay) product; the proposed framework employs a three-term (routing_area $\times$ delay $\times$ power) product throughout this paper.

2 Background and Related Work

2.1 Per-Design eFPGA Layout

This over-provisioning (Section 1) motivates fitting tile placement and fabric dimensions to one specific netlist for a compact, efficient fabric. Applications include IP security (function absent from fixed silicon until configured [7, 22]), domain-specific acceleration, and post-fabrication reconfigurability [20]. Subsequent work reduces overhead by shrinking under-utilized routing [15] or de-regularizing the fabric [9], both motivating the non-island-style layouts we synthesize. All rely on per-design search; none amortizes layout effort across netlists. GOLDS [20] introduced layout search built for a single design: a GA over H-block placement with Area $\times$ Delay fitness through the real VTR flow. A follow-up [4] applies NSGA-II, also co-evolving CLB LUT size and H-block sizing. Both report per-design gains on their own two-term Area $\times$ Delay metric, a different quantity from the three-term ADP we report. Both start from a random population for every new design, transferring nothing from prior runs. We target the same H-block placement problem using an RL approach instead.

2.2 Learned Policies for Physical Design

AlphaChip [18] trains an RL agent to place ASIC macros using a learned proxy and reports cross-design transfer: the amortization argument we make here, at a much larger scale. Subsequent work explores visual representations [17], joint placement-and-routing [6], and transformer policies [16], all supporting the claim that a trained policy amortizes layout cost across designs as per-instance search cannot. Unlike AlphaChip, we evaluate every candidate against the true VTR flow, not a proxy: affordable at our scale, where a VTR run takes seconds to low minutes (Section 4.1).

Within the FPGA setting, RL has guided placement onto an *already-fabricated* grid (annealing perturbations in VPR [11] or a Gym-style VPR environment [5]), but always assigns blocks to a fixed grid, never shapes it. Transferable EDA policies exist for tool tuning [13] and global placement [12]; our policy instead synthesizes the fabric geometry.

2.3 VTR/VPR Background

We build on the open-source Verilog-to-Routing (VTR) [10]: Yosys synthesizes the design [8] and VPR [3] packs, places, and routes against an architecture XML of tile types, positions, and connectivity.

3 Methodology

3.1 Problem Formulation

A fabric optimized for one circuit trades general-purpose flexibility for a significantly smaller, more efficient implementation: it accommodates parameter changes, post-fabrication fixes, and a known set of operational modes on that one primary function, not arbitrary workloads with substantially different H-block requirements.

The *policy* configures the fabric's H-block placement and aspect ratio; the *VTR toolchain* maps the circuit onto it. Given a netlist requiring n_{dsp} DSP and n_{bram} BRAM blocks, the policy emits

$$\pi = \left(\text{ar}, \{(x_i, y_i)\}_{i=1}^{n_{\text{dsp}}+n_{\text{bram}}} \right) : \quad (1)$$

a fabric aspect ratio $\text{ar} \in \mathcal{R} = \{0.1, 0.2, \dots, 2.0\}$, a discrete set of 20 candidate ratios spaced 0.1 apart, and a tile coordinate for every H-block. VTR then synthesizes, packs, places, routes, and auto-sizes the array to the smallest grid of the chosen aspect ratio that holds the design. The policy never places a CLB or fixes grid dimensions; it shapes the fabric and positions H-blocks; the cost is realized through VTR reports. We take that cost to be the three-term ADP (Section 1):

$$\text{ADP}(\pi) = \text{Area}(\pi) \times \text{Delay}(\pi) \times \text{Power}(\pi), \quad (2)$$

where each term is from the VTR flow (Section 4). Area is VPR's *routing area*: since the number of H-blocks placed is fixed by functional requirements, logic block count offers limited sensitivity to placement decisions, while routing area varies directly with H-block positions through their influence on interconnect length and congestion, making it a more discriminative metric for placement quality. Delay is VPR's *critical-path delay*: the longest timing path through the design's logic and routing interconnects. Power is VPR's *total power*: the dynamic and static power dissipated by the synthesized circuit. We report results as percentage ADP reduction relative to a traditional island-style baseline architecture. The Area-Delay-Power Product (ADP) acts as a comprehensive PPA metric, evaluating the design's trade-offs by combining all three physical constraints into a single, joint optimization figure of merit, as expressed in Eq. (2).

3.2 RL Formulation

We pose placement as a finite-horizon sequential decision process and train the policy using MaskablePPO [14], an action-masked variant of the PPO reinforcement learning algorithm, which restricts the action space to legal tile coordinates at each step.

Episode structure. Step 0 selects the fabric aspect ratio from $\mathcal{R}$; each subsequent step positions one H-block (BRAMs then DSPs, both ordered by descending shared-net connectivity). Once all blocks are placed, the environment renders the layout into an architecture XML via Jinja2 [21] and invokes VTR to obtain $\text{Area}(\pi)$, $\text{Delay}(\pi)$, $\text{Power}(\pi)$. Once these metrics are obtained, the final reward is computed by the environment and given to the agent; intermediate steps receive zero reward.

Action space. The action space is Discrete($W_{\text{max}} \times H_{\text{max}}$), fixed across all benchmark circuits, where W_{max} and H_{max} are the core grid width and height taken over both the training pool and the unseen, zero-shot benchmark circuits (Section 5.2). Each benchmark's own *core grid* is its width $\times$ height tile array excluding the surrounding IO ring, sized from its traditional-baseline resource requirements. At step 0, action index a selects $\mathcal{R}[a]$ directly ($a < |\mathcal{R}|$). At every later step, the same index decodes to a tile coordinate $x = 1 + \lfloor a/H_{\text{max}} \rfloor$, $y = 1 + (a \bmod H_{\text{max}})$ within the IO ring, using the fixed H_{max} rather than the active benchmark circuit's real height, so a given index always maps to the same (x, y) regardless of which benchmark circuit is active. Together, the step-0 aspect ratio and the subsequent tile coordinates instantiate the policy output π defined in Eq. (1). Action masking then restricts the policy to indices whose decoded (x, y) falls inside the *active* benchmark circuit's own core grid, fits the block's height without crossing that grid's boundary, and does not overlap an already-placed block; coordinates outside the active benchmark circuit's core grid, even though

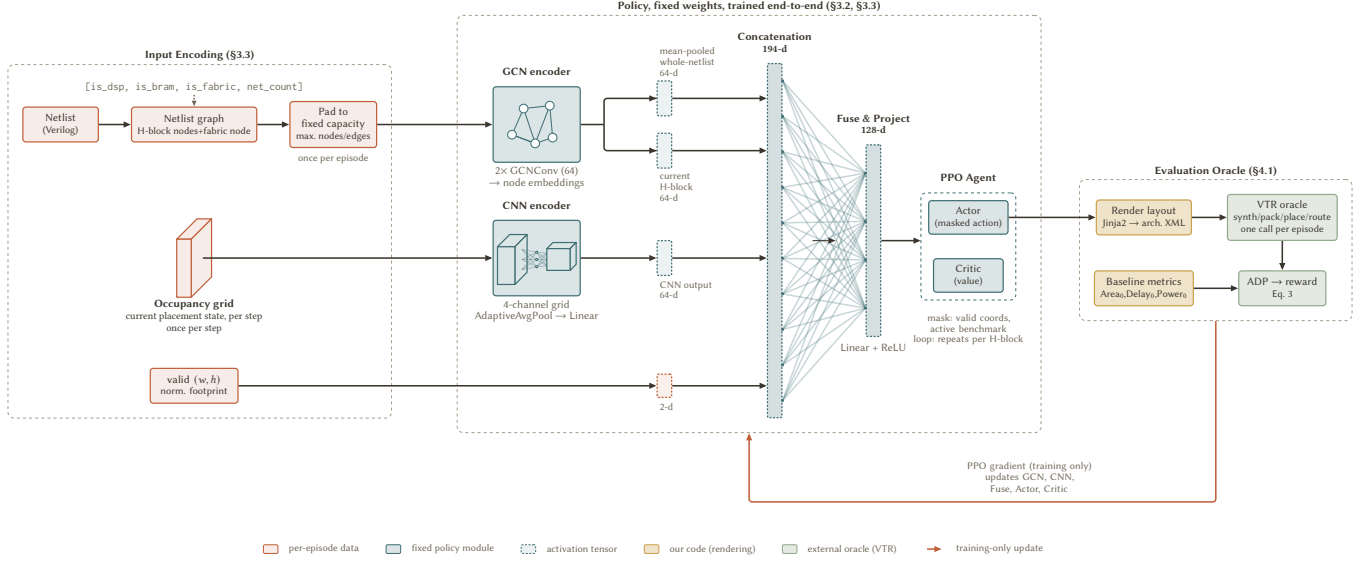


Figure 1: System overview of the RL-based fabric layout search. The left panel encodes the input netlist and grid. The center panel rolls out the placement sequentially, positioning one H-block per step under action masking. The right panel uses Jinja2 to render the final placement into a VTR architecture XML for verification and reward computation.

addressable within the shared $W_{\max} \times H_{\max}$ space, are masked out every step.

Reward. The terminal reward is a sum of negative log-ratios against the traditional baseline for the active benchmark circuit:

$$r = -\left(\log \frac{\text{Area}(\pi)}{\text{Area}_0} + \log \frac{\text{Delay}(\pi)}{\text{Delay}_0} + \log \frac{\text{Power}(\pi)}{\text{Power}_0}\right), \quad (3)$$

where $\text{Area}_0, \text{Delay}_0, \text{Power}_0$ are the traditional-baseline metrics. An invalid layout that fails to complete placement and routing via VTR is terminated early and receives a strict penalty of -10 in place of the reward computed by Eq. (3).

3.3 Multi-Modal Feature Extractor

To generalize across benchmark circuits with varying netlist sizes, H-block counts, and grid dimensions, we encode each netlist as a graph, giving the policy the inductive bias for cross-topology transfer that graph learning provides elsewhere in EDA [23]. We reduce each netlist to a compact fixed-schema graph: one node per H-block instance plus one aggregate “fabric” node for the remaining standard-cell logic, with node features $[\text{is_dsp}, \text{is_bram}, \text{is_fabric}, \text{net_count_normalized}]$. Edges are weighted by shared-net count, normalized to $[0, 1]$, both between H-blocks and between each H-block and the fabric node.

The graph is padded to a fixed maximum node and edge count and encoded by two width-64 graph convolution layers, giving every H-block a learned embedding. Global mean-pooling yields a whole-netlist embedding, and the embedding of the node under consideration at the current step yields a per-step embedding; both are concatenated with a CNN encoding of the 4-channel occupancy grid and the benchmark circuit’s normalized fabric dimensions, then projected to a single 128-dimensional vector consumed by the PPO actor and critic heads. The encoder trains end-to-end under the PPO objective, with no separate pretraining stage.

3.4 Training Setup

Training samples uniformly from the benchmark pool, episode-by-episode, avoiding overfitting to any single benchmark circuit’s topology. We use MaskablePPO, training each run for 13,000 episodes. We report three independent seeds (7, 42, 123) throughout Section 5 to characterize reliability rather than present a single run as ground truth.

4 Experimental Setup

4.1 Evaluation Oracle

Every candidate placement is scored by the true toolchain, not a proxy. A placement is rendered into a VTR architecture XML and VPR constraints file via Jinja2 templates, then run through the full VTR flow (synthesis, packing, placement, routing, and power estimation against a 45nm PTM file), and routing area, critical-path delay, and dynamic power are extracted from VPR reports.² Evaluated pairs are cached; the synthesis-to-routing flow dominates cost (seconds to low minutes per call).

4.2 Benchmark Circuits

Figure 2 lists all 17 benchmark circuits with grid size and H-block counts: 11 for training and 6 held-out for zero-shot evaluation only (Section 5.2), drawn from the VTR suite [19], Koios [2], the wider VTR codebase [10], and two small probes authored for this work, covering the small-fabric end the standard suites underrepresent. Four held-out benchmark circuits (softmax, reduction_layer, arm_core, mkDelayWorker32B) are larger circuits, probing fabric sizes well beyond the training pool.

²All evaluations use VTR 9.0.0 (git 22a09d39, GCC 13.3.0).

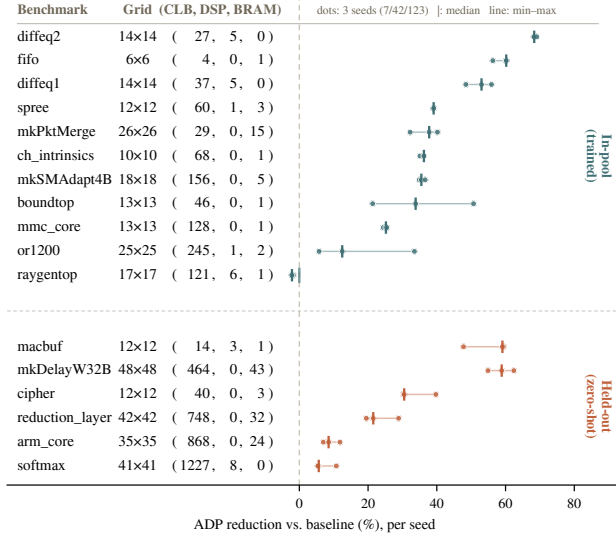


Figure 2: Per-seed ADP reduction for all 17 benchmark circuits (11 in-pool, 6 held-out zero-shot).

5 Results

5.1 In-Pool Performance

Figure 2 (upper group) shows per-benchmark-circuit in-pool best ADP reduction across three seeds. The pooled aggregate (summed best ADP over all 11 benchmark circuits versus summed baseline) is 23.11%, 35.73%, and 23.97% for seeds 7, 42, and 123 respectively, averaging **27.60%**. Ten of the eleven in-pool benchmark circuits demonstrate consistent, positive ADP reduction. The three outliers—raygentop (consistently negative), and or1200 and boundtop (both highly seed-sensitive, spanning 28–29 points)—are all IO-bound circuits where perimeter IO capacity heavily restricts grid dimensions. Their differing outcomes stem directly from interior logic density.

raygentop contains a high density of interior logic (121 CLBs, 7 H-blocks). Selecting an elongated aspect ratio to reduce tile count forces dense interconnects through narrower corridors, which increases necessary channel width. This routing congestion not only negates any physical area savings but also degrades the wirelength-driven power. Consequently, no policy-generated layout outperforms the baseline ADP, yielding stable but negative performance.

Conversely, or1200 and boundtop feature low interior logic density (one to three H-blocks and modest CLB counts). This enables extreme aspect ratios to shrink the physical area without incurring the excessive routing overhead that degrades the power. However, because their improvement depends almost entirely on the agent randomly discovering a specific, narrow range of highly asymmetric aspect ratios rather than optimizing numerous interior block placements, the outcome relies heavily on early exploration dynamics. This single-point sensitivity prevents reliable convergence within a fixed training budget and drives the observed high seed-to-seed variance, contrasting with the more stable results of circuits requiring complex, multi-step spatial optimization.

Table 1: GA average ADP reduction (%) and average VTR calls across three seeds (7, 42, 123), vs. zero-shot policy average ADP reduction (%) over the same seeds, held-out benchmark circuits.

Benchmark	GA avg. ADP red. (%)	GA avg. VTR calls	Zero-shot avg. ADP red. (%)
macbuf	65.33	728	55.46
mkDelayW32B	17.92	746	58.73
cipher	44.13	576	33.49
reduction_layer	7.18	731	23.30
arm_core	17.88	736	9.11
softmax	15.61	784	7.27

As the policy learns, it progressively refines its best-found layout, captured in Figure 3 every time training set a new best-so-far ADP reduction for spree (seed 123). Left of the dotted line, the traditional fabric is shown for reference: it provisions DSP and BRAM columns the netlist never uses. The policy’s best layout overcomes this by keeping only the DSP/BRAM tiles the netlist requires, under an aspect ratio that lets VTR auto-size the array down accordingly, the dominant term in the ADP reduction.

5.2 Zero-Shot Generalization

Figure 2 (lower group) reports ADP reduction on six held-out benchmark circuits across the same three seeds, with zero benchmark-specific training. All six fit within the checkpoint’s fixed dimensions. Four of the six are larger than anything in the training pool (up to 3.4×). Every benchmark circuit is positive under every seed (no sign flips), and every per-seed range is narrow with the overall average being **31.23%**.

5.3 GA Comparison

For a fair point of comparison, we implement a GA under the same oracle, with ADP as its fitness function, implementing the same features as GOLDS over H-block coordinates and aspect ratio. The GA and the policy search in fundamentally different ways; a GA generation is not an RL episode, so we compare the one quantity that is directly comparable between them: the best layout each one finds.

We run this GA baseline across three random seeds (7, 42, 123) on the all six held-out benchmark circuits; Table 1 reports the per-seed average ADP reduction and per-seed average number of VTR calls required by the GA on each, alongside the per-seed average ADP reduction achieved by the zero-shot policy on the same benchmark circuits.

For some benchmark circuits, the GA achieves a greater reduction in ADP than the trained policy, but each of its generations requires its own VTR call, and VTR’s runtime grows substantially with circuit size: on larger benchmark circuits this search cost from scratch grows large. The trained policy instead requires a single VTR call per benchmark circuit at zero-shot inference time, so its relative advantage grows precisely where the GA’s search cost is highest, on large benchmark circuits.

6 Conclusion

We train a single graph-and-grid-conditioned RL policy that places H-blocks and selects fabric aspect ratio, evaluated throughout

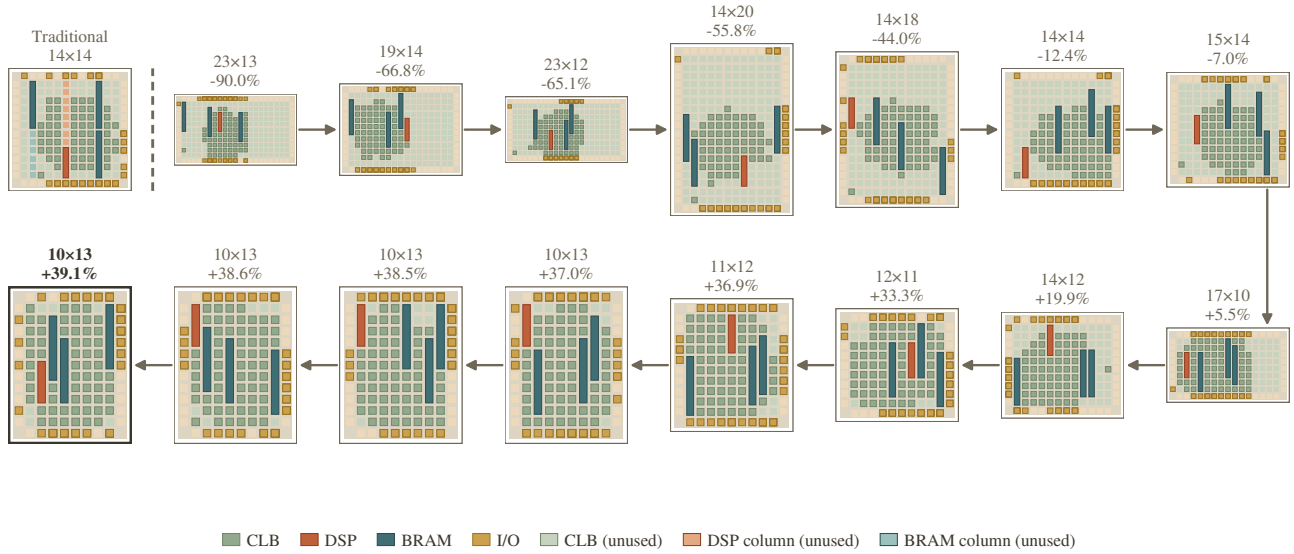


Figure 3: The training progression for spree (seed 123). Arrows trace every episode that set a new best-so-far ADP reduction, left to right across the top row and then right to left across the bottom row. Each panel is a real VPR placement: saturated tiles are occupied, pale tiles are provisioned but unused. Early episodes, still close to random exploration, overshoot badly in both fabric size and ADP. Later episodes converge on a compact, exactly-sized fabric (bottom left).

against the real VTR flow rather than a proxy. The same fixed checkpoint transfers zero-shot to unseen benchmark circuits, including fabrics larger than anything in the training pool, with a positive ADP reduction on every held-out circuit.

This is the practical case for a learned policy over per-instance search. A GA restarts from scratch for every new circuit and incurs its full VTR-evaluation cost each time, a cost that grows with circuit size; the trained policy incurs that cost once, at training time, and reuses it through a single rollout per new circuit. Its advantage therefore grows with the number of circuits the same checkpoint is applied to, and is largest on the large circuits where per-instance search is most expensive. The source code is made freely available at [1] for further use by the research and design community.

References

- [1] Anonymous. 2027. Amortized eFPGA Layout Search via Reinforcement Learning: Source Code, Design Files, and Dataset. <https://anonymous.4open.science/r/PLACEHOLDER-REPO-ID>. Anonymized for double-blind review.
- [2] Aman Arora, Andrew Boutros, Daniel Rauch, Aishwarya Rajen, Aatman Borda, Seyed A. Damghani, Samidh Mehta, Sangram Kate, Pragnesh Patel, Kenneth B. Kent, Vaughn Betz, and Lizy K. John. 2021. Koios: A Deep Learning Benchmark Suite for FPGA Architecture and CAD Research. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 355–362. doi:10.1109/FPL53798.2021.00068
- [3] Vaughn Betz and Jonathan Rose. 1997. VPR: A New Packing, Placement and Routing Tool for FPGA Research. In *7th International Workshop on Field-Programmable Logic and Applications (FPL)*. Springer-Verlag, 213–222. doi:10.1007/3-540-63465-7_226
- [4] D. V. Bhargav, G. Pradyumna, and Madhav Rao. 2025. Meta-Heuristic Optimization of Custom Heterogeneous Blocks Defined eFPGA Design. In *2025 38th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*. IEEE, 326–331. doi:10.1109/VLSID64188.2025.00069
- [5] Ruichen Chen, Shengyao Lu, Mohamed A. Elgammal, Peter Chun, Vaughn Betz, and Di Niu. 2023. VPR-Gym: A Platform for Exploring AI Techniques in FPGA Placement Optimization. In *33rd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 72–78. doi:10.1109/FPL60245.2023.00018
- [6] Ruoyu Cheng and Junchi Yan. 2021. On Joint Learning for Solving Placement and Routing in Chip Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 16508–16519. <https://papers.nips.cc/paper/2021/hash/898aef0932f6aaecda27aba8e9903991-Abstract.html>
- [7] Luca Collini, Jitendra Bhandari, Chiara Muscarelli, Abdol Khader Thakrakattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2025. ARIANNA: An Automatic Design Flow for Fabric Customization and eFPGA Redaction. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 30, 4, Article 63 (2025), 23 pages. doi:10.1145/3737287
- [8] Seyed Alireza Damghani and Kenneth B. Kent. 2022. Yosys+Odin-II: The Odin-II Partial Mapper with Yosys Coarse-grained Netlists in VTR. In *Proceedings of the ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*. 157. doi:10.1145/3490422.3502344
- [9] Voktho Das, Kimia Zamiri Azar, and Hadi M. Kamali. 2026. NuRedact: Non-Uniform eFPGA Architecture for Low-Overhead and Secure IP Redaction. To appear in *Design, Automation & Test in Europe Conference (DATE) 2026*. arXiv:2601.11770 [cs.AR] <https://arxiv.org/abs/2601.11770>
- [10] Mohamed A. Elgammal, Amin Mohaghegh, Soheil G. Shahrour, Fatemehsadat Mahmoudi, Fahrhan Kosar, Kimia Talaei, Joshua Fife, Daniel Khadivi, Kevin E. Murray, Andrew Boutros, Kenneth B. Kent, Jeffrey B. Goeders, and Vaughn Betz. 2025. VTR 9: Open-Source CAD for Fabric and Beyond FPGA Architecture Exploration. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 18, 3 (2025), 39:1–39:53. doi:10.1145/3734798
- [11] Mohamed A. Elgammal, Kevin E. Murray, and Vaughn Betz. 2022. RLPlace: Using Reinforcement Learning and Smart Perturbations to Optimize FPGA Placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (TCAD)* 41, 8 (2022), 2532–2545. doi:10.1109/TCAD.2021.3109863
- [12] Yunbo Hou, Haoran Ye, Shuwen Yang, Yingxue Zhang, Siyuan Xu, and Guojie Song. 2025. TransPlace: Transferable Circuit Global Placement via Graph Neural Network. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*. ACM. doi:10.1145/3690624.3709185
- [13] Hao-Hsiang Hsiao, Yi-Chen Lu, Pruek Vanna-lampikul, and Sung Kyu Lim. 2024. FastTuner: Transferable Physical Design Parameter Optimization using Fast Reinforcement Learning. In *Proceedings of the 2024 International Symposium on Physical Design (ISPD)*. ACM, 93–101. doi:10.1145/3626184.3633328
- [14] Shengyi Huang and Santiago Ontañón. 2022. A Closer Look at Invalid Action Masking in Policy Gradient Algorithms. In *Proceedings of the 35th International FLAIRS Conference*, Vol. 35. doi:10.32473/flairs.v35i.130584
- [15] Hadi M. Kamali, Kimia Z. Azar, Farimah Farahmandi, and Mark Tehranipoor. 2023. Shell: Shrinking eFPGA Fabrics for Logic Locking. In *Design, Automation &*

- Test in Europe Conference (DATE)*. IEEE, 1–6. <https://ieeexplore.ieee.org/abstract/document/10137211>
- [16] Yao Lai, Jinxin Liu, Zhentao Tang, Bin Wang, Jianye Hao, and Ping Luo. 2023. ChiPFormer: Transferable Chip Placement via Offline Decision Transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 18346–18364. <https://proceedings.mlr.press/v202/lai23c.html>
- [17] Yao Lai, Yao Mu, and Ping Luo. 2022. MaskPlace: Fast Chip Placement via Reinforced Visual Representation Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 24019–24030. https://proceedings.neurips.cc/paper_files/paper/2022/hash/97c8a8eb0e5231d107d0da51b79e09cb-Abstract-Conference.html
- [18] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, William Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. 2021. A graph placement methodology for fast chip design. *Nature* 594, 7862 (2021), 207–212. doi:10.1038/s41586-021-03544-w
- [19] Kevin E. Murray, Oleg Petelin, Sheng Zhong, Jia Min Wang, Mohamed Eldafrawy, Jean-Philippe Legault, Eugene Sha, Aaron G. Graham, Jean Wu, Matthew J. P. Walker, Hanqing Zeng, Panagiotis Patros, Jason Luu, Kenneth B. Kent, and Vaughn Betz. 2020. VTR 8: High-performance CAD and Customizable FPGA Architecture Modelling. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 13, 2 (2020), 9:1–9:55. doi:10.1145/3388617
- [20] Pratyush Nandi, Anubhav Mishra, and Madhav Rao. 2024. GOLDS: Genetic Algorithm-based Optimization of Custom FPGA Architecture Layout Design for Secure Silicon. In *Proceedings of the 2024 Great Lakes Symposium on VLSI (GLSVLSI '24)*. ACM, Clearwater, FL, USA, 92–97. doi:10.1145/3649476.3658743
- [21] Armin Ronacher and Pallets Project. [n. d.]. Jinja2. Version 3.1.6. <https://jinja.palletsprojects.com/>
- [22] Chiara Muscari Tomajoli, Luca Collini, Jitendra Bhandari, Abdul Khader Thakkattu Moosa, Benjamin Tan, Xifan Tang, Pierre-Emmanuel Gaillardon, Ramesh Karri, and Christian Pilato. 2022. ALICE: An Automatic Design Flow for eFPGA Redaction. In *59th ACM/IEEE Design Automation Conference (DAC)*. 781–786. doi:10.1145/3489517.3530543
- [23] Nan Wu, Yingjie Li, Cong Hao, Steve Dai, Cunxi Yu, and Yuan Xie. 2023. Gamora: Graph Learning based Symbolic Reasoning for Large-Scale Boolean Networks. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6. doi:10.1109/DAC56929.2023.10247828